Refining Tests through API Response Evaluation

Devika Sondhi
IBM Research
Bengaluru, India
devika.sondhi@ibm.com

Ananya Sharma
IIIT Delhi
Delhi, India
ananya20359@iiitd.ac.in

Diptikalyan Saha
IBM Research
Bengaluru, India
diptsaha@in.ibm.com

Abstract

Most modern web services are delivered via RESTful APIs, making API testing essential for ensuring service reliability. Traditional testing approaches often rely on API specifications, but incomplete or inconsistent specifications produce unrealistic test cases and excessive 4xx errors, lowering test quality. Learning-based methods can infer valid inputs but require large number of requests and responses, limiting industrial applicability. This paper introduces a dynamic test refinement approach that leverages API responses to identify and fix invalid scenarios. By incorporating inferred constraints into the specification through an intelligent agent, the system iteratively refines test cases in a greedy manner. Experiments show reduced 4xx errors and improved test coverage with fewer requests compared to state-of-the-art search-based tools.

ACM Reference Format:
Devika Sondhi, Ananya Sharma, and Diptikalyan Saha. 2026. Refining Tests through API Response Evaluation. In *Innovations in Software Engineering Conference (ISEC 2026), February 19–21, 2026, Jaipur, India.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3796563.3796607

1 Introduction

OpenAPI [29] specification files are often inconsistent with the API's behavior; this may happen due to incorrectly or incompletely written specifications, or these specifications may become inconsistent in the API evolution cycle. Further, certain behavioral constraints in the API may not be structurally supported in the API specification format. For example, the specification cannot capture the associativity between operation parameters. However, knowing such missing constraints would certainly help automated API testers create valid or functional test cases.

The past techniques on API testing have leveraged the input characteristics, such as parameter types, schema structure, and constraints such as minimum, maximum, enum, etc. in the OpenAPI specification to generate the test cases. They have also performed some inferences on API specification to derive important relationships such as producer-consumer relationships [12]. Recent works have also explored utilizing the description tags in OpenAPI specification to apply natural language processing to infer constraints to generate the test cases [21].

However, the limitation of the above approaches is that they heavily rely on the specification to be correct or consistent with the API behavior, resulting in invalid test cases (4xx response code). In such a case, it becomes important to take feedback from the API's behavior to refine the test case generation. An adaptive API testing approach applies reinforcement learning to enhance the exploration strategy in a tool called ARAT-RL [22]. However, these search-based approaches require a large amount of data to learn enough information. This is evident from the large number of API requests that these tools make to be able to attain good coverage.

Figure 1: Sample Response 1

While these techniques may be effective in obtaining good coverage, in the industry scenario, making a large number of API requests incurs a cost that may not be practical [17, 36]. For such tools to be widely used in the practical scenario, the key is to make minimal API requests, while maximizing the testing effectiveness. This effectiveness may be measured through metrics such as operation coverage, defects revealed, and line coverage.

To address this problem, we propose a feedback-driven testing tool, ASTRA (**A**PI's **T**est **R**efinement **A**utomation), which applies a greedy approach to minimize the API requests while focusing on learning from the failures observed on these requests. An underlying intelligent agent in ASTRA, generates test cases intending to create valid (non-4XX) test cases and leverages the natural language response message, along with the response code, for the test refinement. Functional test case constitutes generating a valid sequence of API operations, a valid selection of parameters for each operation, and a valid data value for each parameter.

The main challenge is to understand the response messages and infer constraints in the presence of natural language ambiguities and contextual omissions. The next challenge is to effectively use such constraints along with the existing knowledge derived from specification to create valid sequences, parameters, and data scenarios. The next section discusses our approach and various constraints identified through the initial study and how ASTRA handles each. We use large language models to infer constraints from the response messages and express them in the extended specification model capable of representing all such constraints. This work makes the following contributions:

- To the best of our knowledge this is the first automated black-box API testing system that automatically analyzes the response messages for the generation of valid (that yields non-4xx response code) test cases.
- We present an LLM-based approach to extract various types of constraints from response messages followed by an LLM-based producer-consumer relationship inference and data generation. We believe LLM is well suited for these tasks.

- We present a feedback-driven algorithm that uses response messages and status to generate test cases, while optimising on the API requests made by following a greedy approach.
- By using much lesser responses, ASTRA shows a higher percentage of valid test cases, compared to the existing techniques, in the majority of the benchmarks.

2 Approach

2.1 Overview and Examples

2.1.1 Resolving Parameter Constraints. Fig. 1 shows two types of error response messages received from the two subsequent requests made to the Language-tool API[33]. The first response message indicates that for given optional parameters, exactly one of the two should be given a value. The request includes values for both the parameters, due to which the error 400 was thrown with a message. Note that the OpenAPI specification for this API contains no formal way to specify this inter-parameter constraint; its description states that either of them should be specified but nothing about whether specifying both is valid. So the first challenge is in recognizing the type of constraint from the natural language response. Another challenge is in identifying the entities to which the constraint applies. Among the list of parameters, the challenge is to identify that `text` and `data` are the parameters on which the identified constraint applies. At times, the parameter name may not directly appear in the response message. For instance, the response *'Storage capacity cannot be less than zero'* specifies a minimum value constraint on the parameter `storageCap`. Hence, the second challenge is to automate parameter recognition, given there could be a long list of parameters that the operation takes as input.

ASTRA handles a response error by breaking the test refinement task into these subtasks- error category identification, target identification and actions to derive the constraint class. ASTRA identifies the category of constraints from among a list of categories (Table 2) identified through a preliminary study. This is done by an underlying LLM that learns the definition of various categories and picks the appropriate one for the given error message. The identification of category is essential for ASTRA to determine at what level the test needs to be fixed. The levels could vary from sequence (operation), parameter or data. In the context of the example, one infers that the potential fix is in the parameter scenario of the operation and the identified category is *One(p1, p2...)*. However, ASTRA still needs to identify the arguments (p1, p2) for the constraint. ASTRA then takes an appropriate action for the identified category. The action module is handled by an intelligent agent that allows identifying the target entities and derives a constraint. This results in inferring `text` and `data` as the target parameters. So in this case, an instance of condition class taking the parameters is created as *One(text, data)* and added to the specification model. These constraints are later consumed to generate the test scenarios containing the compliant parameter scenario in the request.

2.1.2 Resolving Data Constraints. On execution of the updated test case derived from the parameter constraint learnt from the previous error, the Language API throws another error in the next run (second message in Fig. 1). The response message indicates that a data value passed in the request is not a supported one. ASTRA

Figure 2: Sample Response 2

identifies the category as a *DataNonArithmeticConstraint(target, value, property)*. The agent of ASTRA populates the constraint to identify that the target parameter is `language` with possible data values 'ar', 'en' etc. as Categorical values. The derived constraint is added to the specification model against the identified parameter. As can be seen at bottom in Fig. 1, using the supported value 'en' for `language` in the subsequent request results in a 200 response.

2.1.3 Resolving Operation Constraints. Consider another example of an invalid request from the Petstore API[30] in Fig. 2. The failed request returning a response 404 in the highlighted block is accompanied by a message 'Order Not Found', indicating a missing `Order` resource. ASTRA categorises this as a *ProducerConsumer* constraint, given a pre-requisite operation was needed to be called to create that `Order` resource. The intelligent agent identifies the consumer parameter, the producer operation and the producer parameter to derive the constraint: *ProducerConsumer(prodOp=placeOrder, prodParam=placeOrder.200.id, consOp=deleteOrder, consParam= deleteorder.path.orderId)* and adds it to the specification model. On the subsequent run, ASTRA derives a sequence of operations `[placeOrder, deleteOrder]` from the constraint and injects the `id` produced by `placeOrder` to `deleteOrder`. This results in a successful 200 response, as can be seen in the figure.

To summarize, ASTRA's approach broadly comprises the following steps:

(1) Categorizing the logged errors into one of the identified constraint types. In total, we have identified 14 categories in which response messages can be classified. 10 of those categories yield constraints that can be used to refine the test cases. Broadly, the constraints may be on the operation, indicating the requirement of a pre-requisite operation; on the parameter level- indicating constraints on parameter occurrence, and on the data level- indicating the parameter input values that should be passed. Such constraints essentially impact three components of a test case - 1) sequence 2) parameter, and 3) data.
(2) Identifying various types of constraints and updating the specification model with the inferred constraints.

(3) Consuming the constraints from the updated specification model to generate sequence scenarios, parameter scenarios, and data scenarios to generate test cases.

2.2 Specification Model

Before explaining the algorithm, we explain the Specification model, which is the data structure of the algorithm. The initial state of the model is obtained by parsing the OpenAPI specification. The specification model extends the OpenAPI specification information by including various constraint types (discussed later) which cannot be formally expressed in the OpenAPI specification. For example, exactly one of the two optional parameters should be passed in the operation request or the value of one parameter should be unique.

The model is denoted as $Spec := \langle O, S \rangle$, where O is the set of operations and S is the set of schemas. Each operation $o \in O$ is a tuple $\langle opname, path, tag, type, IP, OP, GC \rangle$ as described below:

- *opname* is a unique name for the operation. This is typically presented in OpenAPI specification as *operationid*.
- *path* is the relative URL for accessing the operation endpoint.
- *tag* is an optional set of keywords to tag the operation.
- *type* is the HTTP method for RESTful operation that can take values: POST (create), GET (read), PUT (update/replace), PATCH (update/modify), and DELETE (delete).
- *IP* is the set of input parameters (top-level or through transitively unrolled schema). Each input parameter $inp \in IP$ is represented as a 7-tuple $\langle pname, ptype, is\_required, loc, pc, examples, id \rangle$, where *pname* is the parameter name, *ptype* is the parameter type which can be either primitive type(such as integer, boolean, string) or a schema, *is_required* denotes if the parameter is mandatory, *loc* can be one of *BODY*, *PATH*, *QUERY*, *HEADER*, and *FORMDATA*, and *pc* uniformly captures various constraints at the parameter level, such as range, format (email, ipv4), etc. The following section discusses the supported constraint types. *examples* capture any data values given in the OpenAPI specification (present as 'example' or 'examples' attribute), to support using domain-specific values. *id* is a unique identifier of the parameter that is a concatenation of the *opname*, *loc* and *pname*.
- *OP* is a set of output parameters similar to IP but has *responsecode* as an additional property for each parameter. $o.responsecode$ denotes the HTTP response code at runtime.
- *GC* stands for global constraints such as inter-parameter dependencies [24] and inter-operation producer-consumer dependencies. Such dependencies cannot be formally described in the OpenAPI specification, hence the need for a model to support specifying such constraints.

Each schema $s \in S$ is represented as $\langle sname, SF \rangle$ where *sname* represents the schema name and *SF* is the set of fields of that schema represented as $\langle fname, ftype, is\_required, fieldconstraint \rangle$; these are the same as the components of the input parameter.

2.3 Algorithm

The functionality of ASTRA is driven by the following components, as illustrated in Fig. 3.

- Generate and Execute: Use the specification model to generate and run API test cases
- Log Failure: Log endpoints with unique failure responses.

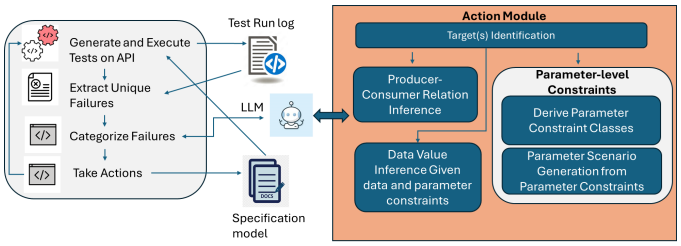

Figure 3: Architecture Design for ASTRA

- Categorize Failure: Identify the failure type from response.
- Take Action: Depending on the error category, obtain appropriate constraint and update the specification model. This invokes a set of sub-actions (depending on the category). This includes selecting the target entity to act on and, if applicable, obtaining the source entity or data value to derive an object of the Constraint class (Table 2).

Note that the loop-back from *'Take Action'* to *Generate and Execute* represents the iterative-learning nature of the approach.

Algorithm 1 explains the iterative learning pipeline of ASTRA. The algorithm consumes the API specification and additional user configuration to produce a specification model and test cases refined over every iteration of learning from the response feedback.

Formally, each test case t is represented as a 5-tuple: $\langle seq, params, data, checks, deps \rangle$ as described below:

- *seq* or sequence scenario is a sequence of operations in the test.
- *params* or the parameter scenario is a mapping between an operation *op* in the *seq* and a set of input parameters $\subseteq op.IP$.
- *data* is the mapping between each parameter in *param* to a value.
- *checks* is the set of checks on the response structure of every operation in *seq*. A typical check for a valid/positive test case is that the response code should be between 200 and 399.
- *deps* is the set of producer-consumer dependencies between the operations in the *seq* where each producer-consumer dependency captures the relationship between input/output parameter of a producer operation with the input parameter of a consumer operation. The test execution engine uses such parameter dependencies to maintain a running state and initialize some input parameters of consumer operations.

The first step of the test generation process generates a specification model from the OpenAPI specification (Function *load_spec*).

Sequence generation. Deriving a valid sequence of operations to execute is important. In every iteration, the latest version of the model is consumed to generate operation sequences to execute later (Function *generate_sequences*). Each sequence *seq* is essentially a target operation preceded by its pre-requisites. For example, to successfully execute `getPetById` operation (target) we need to first execute `addPet` operation which creates the `Pet` resource. The algorithm for sequence formation based on produce-consumer dependency is described later. Initially, as the producer-consumer dependency is empty, the initial list of sequences would contain a single operation in each sequence.

Parameter scenario. To explore diverse combinations of parameters to send in an operation request, we define three types of parameter scenarios. For operations that are pre-requisites, only

Algorithm 1: Iterative testing pipeline

```
Data: API_Spec (JSON file containing the OpenAPI specification),
      Exec_params (a dictionary containing the execution parameters such
      as token to execute the API)
Result: Refined Specification model, TestRuns dump
Function pipeline(API_Spec, Exec_params)
    // Load and Parse specification into spec_model
    spec_model := load_spec(API_Spec)
    learn := True
    failures := set()
    reports := list()
    while learn do
        // Obtain Producer Consumer constraints from the spec_model
        producer_consumer_pairs := extract_dependencies(spec_model) ;
          // initially empty list
        // Obtain list of sequences containing operations preceded by
          pre-requisite operations inferred from producer-consumer
          dependencies
        sequences := generate_sequences(producer_consumer_pairs,
          spec_model)
        // Obtain parameter combinations of each operation - mandatory,
          all, some optional with mandatory
        param_scenarios := generate_param_combinations(spec_model)
        // Generate data for each parameter of operations
        data := generate_data(spec_model)
        // Generate tests for each sequence, parameter, and data scenario
        testcases := generate_tests(spec_model, producer_consumer_pairs,
          sequences, param_scenarios, data)
        // Run tests to log failures and logs
        report, run_failures := execute_tests(testcases)
        // Convergence condition
        if run_failures ⊆ failures then
            learn := False
        end
        failures := failures ∪ run_failures
        spec_model := analyze_failures(spec_model, run_failures)
        reports.add(report)
    end
    return spec_model, reports
end
Data: spec_model (Specification model), run_failures (List of failure
      responses obtained from test run on API)
Result: Refined Specification model
Function analyze_failures(spec_model, run_failures)
    forall failure ∈ run_failures do
        category := classify_failure(failure)
        spec_model := take_action(category, spec_model) ; // returns
          updated spec_model with constraints added
    end
    return spec_model
end
```

a single parameter scenario is generated with all mandatory parameters and for target operation multiple scenarios are generated (Function *generate_param_combinations*): 1) scenarios with a maximal number of parameters (i.e. all parameters), 2) scenarios with the minimum number of parameters (i.e. mandatory parameters), and 3) minimal length scenarios each covering at least one optional parameter. For instance, if an *All* parameters scenario looks as this: *{p1, p2, p3}* and if a constraint exists as *One(p1, p3)*, meaning that only one of p1 and p3 should be present, then instead of All scenario - two scenarios *{p1, p2}* and *{p2, p3}* are produced. The parameter scenario generation algorithm considering different types of constraints that affect it is described later.

Data Scenario. For each parameter, various data values are generated, compliant with all the constraints in the specification model (Function *generate_data*). The data values are randomly generated if no constraint is found on the parameter. For each parameter scenario, a predefined number of data scenarios are generated.

API List		
Car-API	Gestao-Hospitalar	Google-Maps*[19]
Language-Tool	Market	NCS
Person-Controller	Petstore	Problem-Controller
Rest-Countries	RestfulWebService	SCS
User-Management	Xkcd	YoutubeDataApi*[39]
YelpBusinessesSearch*[38]	Stripe*[32]	

Table 1: APIs used to collect responses to study categories.

*Response samples from API documentation were used, limited by the authentication requirements to access the API

Test Generation and Runs. Test cases are generated as a combination of the operation sequence space, parameter structure space, and data space (Function *generate_tests*). All the test case run details are logged, along with capturing all *unique* failure responses (Function *execute_tests*). ASTRA logs all unique responses with 4xx (such as 400, 404) codes to feed to the failure analysis algorithm.

Failure Analysis and Learning. The failure analysis algorithm infers the appropriate constraints to add to the specification model, such that the error response could be avoided in the following iterations of the test case generation and execution (Function *analyze_failures*). This approach aims to obtain valid test cases, especially to minimize the 4xx responses, which are indicative of invalid requests at the API user's end. On the other hand, 5xx responses are unhandled errors at the API's end and usually reveal defects in the API. By learning from the 4xx failures, any subsequent 5xx would be a step closer to revealing defects in the API. The algorithm converges when no *new* failure is logged from the run.

The analysis contains three steps: 1) classify the response to one of the constraint categories, 2) create the constraints by identifying the entities associated with the constraint types, and 3) take action on the identified entities based on the identified constraint category which may involve adding the constraints to the specification model and generate test data with them, ask for user-input typically to obtain authentication/configuration input or ignore. These steps have been described in the following section.

2.3.1 Constraint Inference.

Constraint Categorization. We conducted a short study to obtain the category of constraints observed in API responses. We collected response messages from over 17 APIs (Table 1) when run with API testing tools MOREST [23], EvoMaster [10]. For 4 of these APIs, the tools could not be run due to usage limits. For these particular APIs, we have studied their API documentation to collect response messages to include in the dataset. Additionally, the response data used by Lopez et al. in their analysis work on inter-parameter dependencies [24] was also included in the dataset. This resulted in a dataset size of 17,887 response instances. On extracting the unique responses, we obtained a dataset of 1663 responses. These response messages were studied to identify 14 different types of categories. The categories are described in Table 2.

Except for categories 1, 3, 9, and 14, all other categories are expressed as constraints in the specification model. The OpenAPI does not allow formally specifying categories 5, 6, 7, 8, 12, and 13 and hence, we designed our specification model to accommodate such constraints. Constraints in categories 2-3 impact the sequence

	Category	Description	Sample
1.	Configuration /Authentication	Data is required from user, such as credentials	*API key not valid. Please pass a valid API key.*
	Operation-level		
2.	ProducerConsumer (producerop, producerparam, consumerop, consumerparam)	A pre-requisite operation exists for the target operation	*playlistNotFound: The playlist with the ID 'playlist456' could not be found.*
3.	UnsupportedOperation	Invalid operation present in specification	*Request method 'POST' not supported*
	Parameter-level		
4.	AdditionalMandatory Parameter (p1.is_required=true)	Parameter p1 must be present	*"points" is a required parameter.*
5.	Or(p1, p2,.., pn)	If optional parameters exist, at least one should be specified	*You must specify either the 'source' or 'destination' parameter.*
6.	One(p1, p2,. . ., pn)	Among a subset of parameters only one should be given value	*Either city or zipcode is required, not both.*
7.	AllOrNone(p1, p2,.., pn)	Among a subset of parameters, either all should be given value or none specified.	*Address should be specified with street, city and pincode.*
8.	ConditionalParameter Required (p1, isPresent1, p2, isPresent2)	If param p2 present/absent then param p1 should be present/absent;	*If longitude specified then latitude should be too*
9.	Parameter-Unknown	Request has an unknown parameter	*Received unknown parameter: url*
	Data-level		
10.	Data-Arithmetic (p1, reloperator, p2/const)	p1 is Greater than, less than, equal, unequal to p2 or a constant/list of constants	*afterTimestamp must be greater than beforeTimestamp*
11.	Data-NonArithmetic (p1, property, value)	Invalid data based on type, uniqueness	*'PL' is not a valid gender. Supported values are 'Male', 'Female', 'Other'.*
	Parameter+Data		
12.	DataInfluenced-ParamSelection ($DataArithmetic \implies PC$)	Parameter selection constraints based on data values	*If type is 'audio', only one of the other two parameters is required*
13.	ParameterInfluenced Data Values ($PC \implies DataArithmetic$)	Constraint on Data values if param1 is present/absent	*If thumbnail is present, type must be 'link'.*
14.	Unhandled	Constraint can't be determined. Should be reported as potential defect	*Internal Server Error*

Table 2: Constraint Categories.

scenario formation, categories 4-9 are related to parameter scenario formation and solely dependent on the parameter properties, and categories 10-11 impact the data scenario formation and are solely dependent on data values. Categories 12-13 are the interesting ones as they are conditional to the constraints in other categories.

Mapping natural language response messages to the above categories is modeled as a classification problem. We have used an LLM to retrieve the constraint types. We use a zero-shot prompt for the Mistral Large-2 model to infer the categories. When prompting to LLM model cannot identify a category (responds with *'Unhandled'* category), we use an alternate fine-tuned classification model. We trained a classifier using a BERT-based transformer model [16], on the response message data to classify a concatenated string of response message and response code to one of the categories described in Table 2. With a train:test split of 80:20, we trained for 5 epochs to obtain the classification accuracy of 95.5% on the test set. With this, we obtained the classification model to use in ASTRA.

Constraint Identification. Once the response message is classified into a category, ASTRA uses its `Action Module` to form the specification constraint objects. Its underlying implementation has an intelligent agent based on Mistral-Large model performing tasks such as entity recognition, data generation etc., depending on the constraint category. The goal of the agent is to derive the constraint object and populate to the specification model at an appropriate point. Table 2 shows the format of the specification constraint along with the categories. The *PC* (parameter constraints) in categories 12 and 13 denotes any constraints from categories 5, 6, 7, and 8.

Even though we implemented separate functions to form constraints in separate categories, there are two common challenges associated with the constraint formation process: 1) identification of entities associated with the constraints, 2) disambiguation of parameter positions in arithmetic and logical relations, and 3) identification of nested constraints for categories 12 and 13.

We now discuss the challenges and solutions to infer the constraint entities. These entities could be the API operation, parameter, property of parameters, or constant values. The first problem is to match parameter names with natural language variations. Consider the response message *'Internal Server Error: This email*

beulalingo@yahoo.com is already in use.'; from among several candidate parameters *['gender', 'linkedin', 'password', 'secured', 'mobile', 'website', 'address', 'username', 'name', 'emailId', 'country']* for the operation, it should pick *'emailId'* to act on. ASTRA infers this entity, using prompt to the LLM by populating it with the dynamically obtained information, such as parameter names of the operation and error message for the given example.

Consider the expected constraint ProducerConsumer(producerop= *addPet*, producerparam=*id*, consumerop=*createOrder*, consumerparam=*petid*) and a response message 'Pet not found' from PetStore API while testing *createOrder* operation (POST operation) for creating an order related to a pet. The AI agent consumes information about the 1) identified target (consumer parameter) from the response message and 2) list of potential producer operations (POST type operations) and their parameters to infer the ProducerConsumer pair. The agent intelligently identifies the producer as the post-operation, which is closely associated with the noun entity (Pet) in the message. Producer's id in output param becomes the *producerparam*. The consumer's input param *petId*, closely associated with the combination of the producer's resource name (Pet) and producer parameter name (id), is obtained as *consumerparam*.

Data-level constraints are similarly handled by the agent by consuming the target and the response error to producer set of realistic values to use as test data.

The second challenge pertains to identifying and encoding logical relations between parameters from natural language. For instance, an error message: *'gain should surpass expenditure'* can be expressed logically as *gain* > *expenditure*. The latter is easier to analyze to generate data values when encoded as constraint ConditionalParameterRequired(p1= *gain*, reloperator= >, p2= *expenditure*). To obtain such an expression, the underlying agent consumes the response and the list of identified target parameters.

To identify the nested constraints in categories 12 and 13, we break the sentence into two parts corresponding to the antecedent and consequence of the implication. To identify the *PC* part of the constraints, we use the relevant part to classify the type of parameter constraint and then use the relevant constraint formation procedure for specific parameter constraint categories.

Actions and Scenario Formation. Once all the categories are identified with relevant constraints, there are different types of actions taken based on the categories. For the Configuration/Authentication (1) category user is asked to provide the configuration details. The unsupported operations (3) and unknown parameters (9) are deleted from the specification. Unhandled cases (14) are reported as errors, performing no further processing. The rest of the cases influence the sequence, parameter, and data scenario formation. Hence, we describe inference of constraints from such formation.

Sequence Scenario. As stated earlier sequence scenario generation uses producer-consumer dependencies to form pre-requisite for the target operation. For each operation (target), we identify its prerequisites by transitively traversing the producer-consumer dependency with consumers as known. If for an operation multiple producers are found then only a single producer is used which generates a single resource. Note that, we have seen multiple producers are possible for a resource/schema when one producer generates one resource and another producer generates multiple resources of the same type. For testing a consumer operation, not all its producers are required. All the operations are topologically ordered.

Parameter Scenario. The parameter scenario generation algorithm expresses all parameter-selection constraints into Z3- constraints. Each parameter gets either 1 or 0 value correlating whether it will be selected or not. All mandatory parameters get 1 value. If any optional parameter of a schema is 1 then the mandatory parameter of the schema is also 1. If all optional parameter list is OL, then $Or(p_1, \dots p_n)$ constraint is expressed as of $sum(p \in OL) > 0 \implies sum(p_1, \dots, p_n) > 0$. $One(p_1, \dots, p_n)$ is expressed as: $sum(p_1, \dots, p_n) > 0 \implies sum(p_1, \dots, p_n) = 1$. $AllOrNone(p_1, \dots, p_n)$ constraint is expressed as $sum(p_1, \dots, p_n) = 0 \vee sum(p_1, \dots, p_n) = n$. $ConditionalParameterRequired(p1, b1, p2, b2)$ is expressed as $p2 = int(b2) \implies p1 = int(b1)$. The challenging case is DataInfluenced-ParamSelection. Since parameter scenarios are generated before data scenario generation, we divide this constraint into two: one where all parameters related to DataArithmetic parameter are present (i.e. add up to its size) along with the existing PC constraint, and another case where this constraint is omitted.

The algorithm repeatedly calls constraint solver to obtain different parameter scenarios and then filters to get parameter scenarios with maximal length, and minimal length, and iteratively finds a minimal length scenario covering each optional parameter.

Data Scenario. Given a parameter scenario, we first gather all the relevant data constraints containing all parameters in the scenario. If $ParameterInfluencedDataValues(PC \implies DataArithmetic)$ exists, then *PC* is evaluated against the parameter scenario, and if it holds then *DataArithmetic* constraint is also gathered. If $Data\ Influenced - ParamSelection(DataArithmetic \implies PC)$ is present, *DataArithmetic* constraint is also gathered if *PC* evaluates to true. To generate realistic values satisfying all the constraints, we create a prompt expressing all the constraints in natural language and query llama-2-7b model [34] to generate the required number of data scenarios. The resultant scenarios are checked against the gathered data constraints (using a Z3 constraint solver), and if not satisfied, then the Z3 solver is used to generate satisfiable values.

Handling Blank Response. To handle scenarios of blank response messages, which were found in 22% of the requests over the benchmark APIs, ASTRA additionally leverages the response code to infer possibility of a 4xx. It derives heuristics to handle such responses. For instance, 404 response is a *'Not found'* scenario and if the operation under test is found to be requiring and identifier in its input, then the case is treated to be a ProducerConsumer Constraint. The LLM is responsible to identify if any of the input parameters represent an identifier. Other 4xx blank responses are assumed to be a data issue and the subsequent data values for such operations are generated by the agent. Applying these heuristics, ASTRA was able to resolve several 4xx responses even in the absence of a response message. Nevertheless, the content of the response is certainly more useful in making the correct refinement to the test case.

Table 3: Benchmark services used in the evaluation.

API	Operations	GET, POST, PUT, DEL
petstore [30]	20	8, 7, 2, 3
person [2]	12	5, 2, 2, 3
genome [2]	23	13, 10, 1, 0
gestao [20]	20	10, 5, 3, 2
user mgmt. [2]	23	6, 7, 2, 5
market [2]	13	7, 2, 3, 1
problem cont. [2]	8	2, 3, 1, 2
langtool [33]	2	1, 1, 0 , 0
scs [2]	11	11, 0, 0, 0
ncs [2]	6	6, 0, 0, 0
rest countries [2]	22	22, 0, 0, 0

3 Experimental Evaluation

This section presents the evaluation of ASTRA to investigate the following research questions.

3.1 Research Questions

- *RQ1*: How effective is ASTRA based on coverage metrics?
- *RQ2*: On effectiveness of learning with functional test cases
  - *2a* How effective is ASTRA in reducing invalid test cases with every iterative learning?
  - *2b* How effective is ASTRA in generating valid test cases?
- *RQ3*: How does ASTRA perform with respect to operation hits made, as compared with the state of the art tools?
- *RQ4*: How effective is ASTRA in revealing API defects?

3.2 Experimental Setup

All experiments were run on a machine with Microsoft Windows 11 OS, with 32 GB RAM and 11th Gen Intel Core i7-1165G7 2.80GHz processor. The detailed results and the runnable wheel of ASTRA have been shared on the repository [1].

3.2.1 Benchmarks. We selected 11 public APIs, that have been used in the past research ([2, 5, 22]). These 11 APIs were short-listed applying a similar elimination strategy as was applied for ARAT-RL[22]. We eliminated APIs that required authentication, had external dependencies on services or had usage rate limits. We used these as benchmarks to evaluate ASTRA. Table 3 lists the benchmarks, along with details of number of operations and distribution of operations over various REST calls.

3.2.2 Tools. To investigate the first three RQs, we compare ASTRA with three state-of-the-art API testing tools, ARAT-RL [22], EvoMaster (v1.6.0) and MOREST [23][10]. These three tools have been shown to be the best performing API testing tools in the recent paper by Kim et al. [22]. Considering, ASTRA is a blackbox tool, we use the blackbox mode of EvoMaster for a fair comparison. These tools have been executed with their default configurations, setting a time budget to 30 minutes, wherever applicable. ASTRA does not have a time budget as its termination condition is when no error or previously seen errors occur in the next iteration. Note that the time budget set for the other tools is more than the maximum of 16 minutes (average 5 minutes) run time seen over one of the benchmarks by ASTRA. Furthermore, for a fair evaluation of ASTRA, that focuses on the effectiveness of API testing with minimal requests, we also executed ARAT-RL, EvoMaster and MOREST with an operation hit budget of the number of requests made by ASTRA for each benchmark API. This budget was obtained from averaging over 3 runs of ASTRA on each benchmark API. To summarise, each of the three benchmarks tools has been executed in two modes: 1) time budget of 30 minutes, and 2) hit limit *(hl)* of requests made by ASTRA for the corresponding benchmark API. The presented results have been averaged on 3 runs of each tool.

The code of MOREST and ARAT-RL was altered by adding hit limit constraint to the loops. For EvoMaster, the `maxActionEvaluations` and `stoppingCriterion` parameters were passed to impose this limit, as suggested by one of the authors of the tool.

3.2.3 Result Collection. Data collection for code coverage was done using JaCoCo[1]. Other metrics of operation coverage, 2xx, 4xx, 5xx response were extracted from the output reports produced by the tools, using a python script we wrote for each of the tools. The results are available as supplementary material.

3.3 Results

MOREST could not be executed on the *user* API as it terminated with a 'keyError'. ARAT-RL could not be executed on *problem* API as the specification parsing failed due to recursive schema references. EvoMaster and ASTRA were successfully executed on all APIs in the benchmark. Table 4 indicates the total number of requests made by each tool, without a hit limit, over benchmarks.

Table 4: Operation hits across tools.

API	ASTRA	MOREST	EvoMaster	ARAT-RL
petstore	115	7880	50	41,600
person	42	46,199	23	53,043
genome	157	1947	50	17,631
gestao	186	5903	60	16,800
user	134	-	60	44,577
market	132	19,676	26	19,536
problem	68	7156	19	-
langtool	37	20	2	247
scs	11	64,216	12	72,148
ncs	6	31,059	16	89,888
rest countries	37	4620	25	73,734
	925	188,676	343	429,204

3.3.1 ***RQ1: Performance on Coverage Metrics.*** We computed 4 coverage metrics for the 4 tools across the benchmark APIs- line coverage, branch coverage, operation coverage and successful operation coverage. The line coverage (*lc*) and branch coverage (*bc*) have the standard meaning. Operation coverage (*oc*) measures the percentage of operations successfully executed, with 2xx or 5xx response, with respect to the total operations in the API specification. We further computed a successful operation coverage (oc_{2xx}) that measures the percentage of operations successfully executed, with 2xx response, with respect to the total operations in the API specification. Except for ASTRA, additional metrics, oc_{hl} and oc_{hl2xx}, were computed for the remaining three tools, representing the two types of operation coverage observed when the hit limit budget was imposed. For 4 APIs, Jacoco results showed a 0% line and branch coverage. This seemed incorrect as responses from the API were

[1] https://anonymous.4open.science/r/ASTRA-B96F

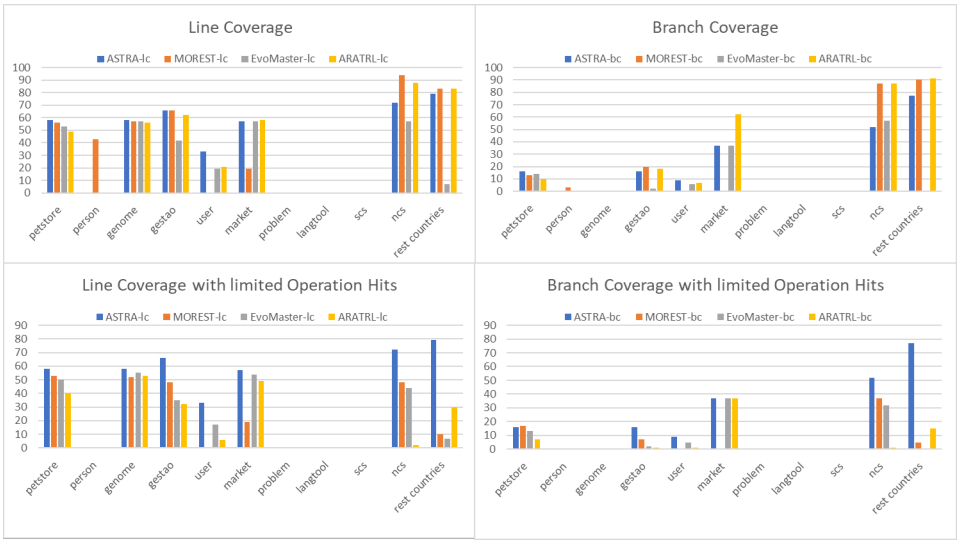

Figure 4: Line and Branch Coverage of 4 tools across 11 APIs, with and without OpHit Limit

Table 5: Operation Coverage Metrics with and without hit limit budget; values in percentage.

API	ASTRA		MOREST				EvoMaster				ARAT-RL			
	oc	oc_{2xx}	oc	oc_{2xx}	oc_{hl}	oc_{hl2xx}	oc	oc_{2xx}	oc_{hl}	oc_{hl2xx}	oc	oc_{2xx}	oc_{hl}	oc_{hl2xx}
petstore	**100**	90	75	75	20	15	75	55	60	55	50	50	20	15
person	**100**	25	92	58	33.3	0	83.3	25	83.3	25	83.3	58.3	17	8.3
genome	**100**	100	96	91.3	35	35	65	65	61	61	96	96	56.5	56.5
gestao	**90**	85	90	90	40	35	25	25	0	0	80	80	20	15
user	**87**	61	-	-	-	-	52	30	39	17.4	74	52.2	30.4	13
market	54	15.4	100	0	85	0	38	15.4	31	8	62	15.4	23.1	0
problem	50	38	63	38	50	25	63	38	63	38	-	-	-	-
langtool	**100**	100	50	50	50	50	50	50	50	50	100	100	50	50
scs	**100**	100	100	100	9.1	9.1	100	100	91	91	100	100	27.3	27.3
ncs	**100**	100	100	100	100	100	100	100	83.3	83.3	100	100	0	0
rest countries	**100**	100	100	100	0	0	0	0	0	0	100	100	9.1	9.1

observed. We were unable to resolve this and hence, indicate these metrics as unknown for *person, problem, langtool* and *scs*.

Fig. 4 shows the line and branch coverage trends for all the 4 tools. Over the 7 APIs for which the line coverage could be obtained for all 4 tools, while ASTRA outperforms for 3 APIs- petstore, genome and user, it was observed at par with the best performing tools for 2 APIs- gestao and market. It did not perform the best on the branch coverage metric; this could be attributed to the reason that ASTRA performs a focused test generation, driven by response-feedback, to reduce 4xx responses. As a result, it may not generate diverse data for operations covered successfully, as it is better suited for directing towards functional tests. However, with the hit limit, ASTRA emerges as the strongest, indicating its efficacy in learning with minimal data seen from the responses.

Table 5 presents the operation coverage metrics in percentage, as computed over 11 benchmarks for the 4 tools. It indicates that without imposing a hit limit on the benchmark tools, ASTRA outperforms or is at par with them on *oc* for 9 APIs. The reason why it missed certain operations on the remaining 2 APIs is that other tools were able to obtain a 5xx response on the operations missed by ASTRA through random data and invalid parameter scenarios. The focus of ASTRA is to drive tests towards valid input in order to highlight more meaningful potential defects, it minimizes invalid tests and hence, misses on these operations resulting in a 5xx. To fairly evaluate how many of the operations were covered through valid tests, we also captured coverage of operations resulting in a 2xx response. ASTRA also performs the best over oc_{2xx} for 9 APIs. An interesting case of market API indicates that while *oc* is highest for MOREST, it has the lowest oc_{2xx}, indicating all 5xx responses obtained by it. On investigating, all the error messages indicated "internal server error", which could be attributed to the random

nature of data values. For market API, oc_{2xx} for ASTRA was seen to be at par with the best performing tool.

As evident from Table 4, MOREST and ARAT-RL make significantly more requests than ASTRA. Making thousands of hits can be a practical obstacle for industry APIs owing to quota limits and computational expense, which incurs a cost[17]. Hence, we evaluated how quick these tools are in exploring the search space with a hit limit. As indicated by the oc_{hl} and oc_{hl2xx} in Table 5, ASTRA emerged as a clear winner, with its *oc* being the highest for 10 of the 11 APIs and oc_{2xx} being the highest for all APIs. The operation coverage metrics fell sharply, especially for MOREST and ARAT-RL, on applying this budget. This is indicative of reinforcement learning-based approach being impractical for industry API testing. To summarize, ASTRA achieves the best coverage with practically feasible number of requests. This is attributed to its feedback-driven greedy approach that learns faster from limited responses.

3.3.2 ***RQ2: Effectiveness of Learning.*** Fig. 5 indicates how the proportion of 2xx, 4xx, 5xx responses over total responses change with every iteration of ASTRA. The overall trend shows that the 2xx, 5xx increase, while, 4xx responses decrease. The change seems significant for 5 APIs (petstore, language, gestao, problem and user) and no change for 6 (scs, ncs, market, genome, person and rest-countries). Gestao and petstore contain several producer-consumer dependencies which are learnt from the response with each iteration. *langtool* contained various inter-parameter dependencies and data-value constraints that are learnt from response messages to refine the parameter scenarios with each iteration. Two APIs, *market* and *person*, resulted in 5xx response covering all operation in the first run. *scs*, *ncs*, *genome* and *restcountries* covered all operations with 2xx responses in the first run, hence, deriving no learning. To summarize, the overall observations show 1) a reduction in invalid test cases and 2) an increase in valid test cases, indicating the effectiveness of learning from response messages in refining the test cases for functional testing.

3.3.3 ***RQ3: Operation Hits.*** The ideal case would be to make minimal operation hits, while maximizing the coverage and defect revelations, especially in an industry API that may be performing computationally intensive services. Fig. 6 shows the proportion of various response status received across the benchmarks by all 4 tools, without and with hit limit(suffixed as HL). Even with a much lesser operation hits as compared with MOREST and ARAT-RL, ASTRA was able to produce better proportion of 2xx responses than ARAT-RL and EvoMaster, while not very far behind MOREST. With *HL* mode, ASTRA defeats all 3 tools. With respect to the minimal proportion of 4xx responses, a similar performance rank was observed. Accounting for 2 APIs *(user, problem)* that could not be executed by some benchmark tools, we omitted these 3 APIs and recomputed the numbers that can be found in the supplementary material. The same trend was observed.

3.3.4 ***RQ4: Defect Detected.*** To evaluate the potential of ASTRA in revealing defects, we gathered the count of unique 5xx responses obtained by it across APIs. As MOREST received the maximum proportion of 5xx responses, we select MOREST to compare our defect results against. Table 6 indicates the unique defects obtained across the benchmark operations by both tools, with and without the hit limit applied. ASTRA revealed more number of defects than MOREST, in both the settings. Of the 33 defects revealed by ASTRA, 13 contained the stack-trace, which can be deemed to be serious in nature. MOREST revealed 10 such defects containing stack-trace. In our future experiments, we would be interested in removing the termination condition on ASTRA to continue running for a time budget to explore if the defect revealing capability improves. The current numbers are, nonetheless, indicative of the effectiveness, as the defect revealing 5xx responses were observed to be better.

Table 6: Unique Defects detected by ASTRA and MOREST.

API	# Defects by ASTRA	# Defects by MOREST	
		w/o Hit budget	With Hit budget
petstore	3	0	1
person	11	10	4
genome	0	1	0
gestao	1	2	2
user	6	0	0
market	8	13	11
problem	1	2	2
langtool	2	0	0
scs	0	0	0
ncs	0	0	0
rest countries	1	1	0
	33	29	20

In conclusion, ASTRA performed the best on coverage metrics and in generating valid test cases, even with limited API requests. The iterative approach indicated effectiveness in reducing the invalid test cases, taking the testing technique closer to exploring defects in the APIs. These results make it suitable for industry usage.

3.4 Threats to Validity

While the results on ASTRA have been computed with the termination criteria enabled, a larger time budget was set for other tools. This poses an internal threat to validity. The presented approach aims to perform test refinement with optimizing the operation hits; however, taking liberty on making a higher number of operation hits may also end up revealing more defects in APIs and a better line and branch coverage. Nevertheless, even with the request-conservative, approach we see promising results that are better or at par with benchmark tools.

External threats may come from the choice of LLM model for certain components of ASTRA, such as classification and data constraint learning. For every respective component requiring LLM, the choice of model was made from among SOTA openly available models giving stable and best results, as assessed on 3 APIs randomly picked from the benchmark with over 3 runs of ASTRA on each. The metrics along each run remained in close proximity. As in the future, improved models may be released, this would only complement the performance of ASTRA. The results may not be generalized, posing an external threat to validity. Although diverse APIs were picked from the past literature on API testing, we plan to evaluate ASTRA over industry APIs.

4 Related Work

Rest API Testing techniques can be broadly classified into black-box and white-box. As our technique is black-box, we focus on black-box API testing papers.

Atlidakis et al. [11, 12] created RESTler, which creates operation sequences from the producer-consumer relationship and explores

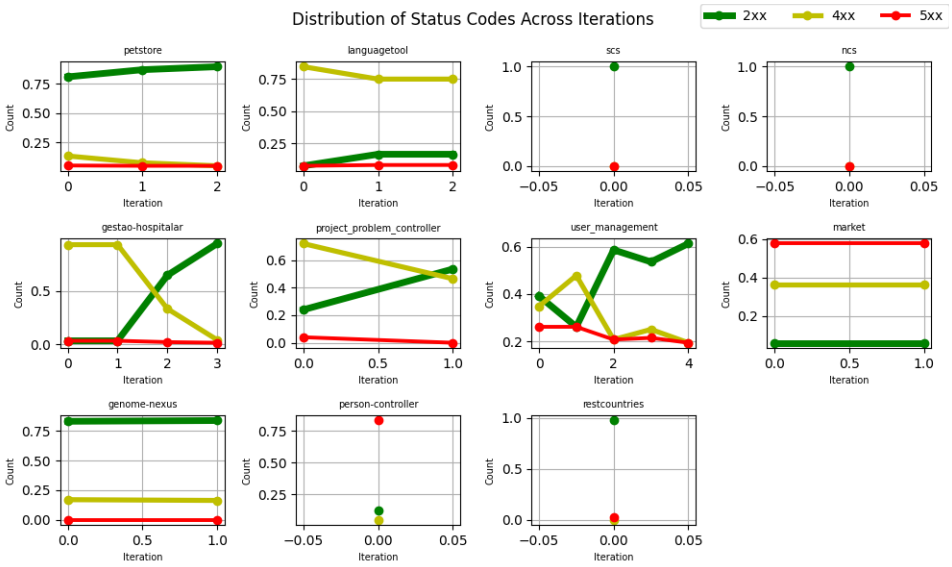

Figure 5: Trend of Response Code with each Iteration of Refinement with ASTRA

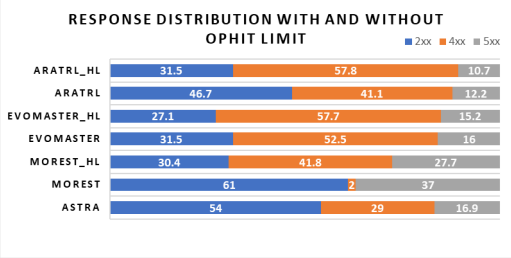

Figure 6: Response Code Distribution

the operations in a breadth-first fashion. Their sequences do not maintain any resource life-cycle relationship. As identified in [23], their producer-relationship contains many false positives. The authors extended RESTler with intelligent data fuzzing [18].

Arcuri et al. [3–10, 27] developed EvoMaster which includes novel techniques for test case generation for API with white-box access using a genetic algorithm. They also have a black-box version of the algorithm [8] which essentially performs random testing adding heuristics to maximize HTTP response code coverage.

RestTestGen [15, 37] infers parameter dependencies with better field name matching than just plain field-name equality which uses field names and object names inferred from operations and schema names. They maintain the CRUD order and create a single sequence to test all the operations for *all* resources in the API. It uses a response dictionary and uses examples and constraints in API spec and random values if such information is not available.

Giamattei et al. [31] proposed a black-box testing approach, uTest, which uses a combinatorial testing strategy [28]. The input partition for all parameters is formed based on [13] and divided into valid and invalid classes. It uses examples and default values from specifications and bounds to generate random values. They perform test case formation for each operation. No details about the producer-consumer relation or sequence formation are mentioned.

RESTest [26] uses fuzzing, adaptive random testing, and constraint-based testing where constraints are specified manually in an OpenApi extension language, especially containing the inter-parameter dependency. Their sequence generation is confined to each operation with the required pre-requisite. Corradini et al. [14] implemented the black-box test coverage metrics defined in [25] in their tool, Restats, which performs operation-wise test case generation. Tsai et al. [35] use the test coverage [25] as a feedback to black-box fuzzers. They do not produce resource-based functional test cases but follow the order of test operations based on paths and CRUD order. They also use the concept of ID parameters.

MOREST [23] captures the following dependencies - 1) operation o returns an object of schema s, 2) operation o uses a field or whole of the schema s, 3) operation o_1 and o_2 are in the same API 4) two schema s_1, s_2 which has a field in common in the form of resource-dependency property graph. It uses the graph to generate the test cases and rectify the property graph online. ARAT-RL [22] performs API testing by using a reinforcement learning-based approach to incorporate response feedback to prioritize the operations and parameters in its exploration strategy. The approach aims to maximize coverage and fault revelation. While the approach uses the response code to update the reward and response data to derive data values, it misses processing the information-rich error messages to incorporate into its search approach.

5 Conclusion

We presented a novel black-box API testing algorithm that uses the response message to generate functional test cases. Our objective is to create a reasonable number of valid test cases giving high coverage. Central to our technique is the extraction of constraints from response messages using an intelligent agent. We augment the specification model with the constraints and generate satisfying test cases. Our technique performed the best compared to existing techniques when evaluated on 11 APIs. ASTRA generated valid test cases with a small number of requests due to an intelligent procedure and therefore suitable when API hit is costly.

References

[1] [n. d.]. EclEmma - JaCoCo Java Code Coverage Library — eclemma.org. https://www.eclemma.org/jacoco/. [Accessed 20-10-2023].

[2] 2022. RestGo repository. https://github.com/codingsoo/REST_Go. [Online; accessed 19-August-2022].

[3] Andrea Arcuri. 2017. Many independent objective (MIO) algorithm for test suite generation. In *International symposium on search based software engineering*. Springer, 3–17.

[4] Andrea Arcuri. 2017. RESTful API automated test case generation. In *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 9–20.

[5] Andrea Arcuri. 2018. Evomaster: Evolutionary multi-context automated system test generation. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 394–397.

[6] Andrea Arcuri. 2018. Test suite generation with the Many Independent Objective (MIO) algorithm. *Information and Software Technology* 104 (2018), 195–206.

[7] Andrea Arcuri. 2019. RESTful API automated test case generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 1 (2019), 1–37.

[8] Andrea Arcuri. 2020. Automated black-and white-box testing of restful apis with EvoMaster. *IEEE Software* 38, 3 (2020), 72–78.

[9] Andrea Arcuri and Juan P Galeotti. 2021. Enhancing search-based testing with testability transformations for existing APIs. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–34.

[10] Andrea Arcuri, Juan Pablo Galeotti, Bogdan Marculescu, and Man Zhang. 2021. EvoMaster: A search-based system test generation tool. *Journal of Open Source Software* 6, 57 (2021), 2153.

[11] Vaggelis Atlidakis. [n. d.]. Testing of Cloud Services. ([n. d.]).

[12] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 748–758.

[13] Antonia Bertolino, Guglielmo De Angelis, Antonio Guerriero, Breno Miranda, Roberto Pietrantuono, and Stefano Russo. 2020. DevOpRET: Continuous reliability testing in DevOps. *Journal of Software: Evolution and Process* (2020), e2298.

[14] Davide Corradini, Amedeo Zampieri, Michele Pasqua, and Mariano Ceccato. 2021. Restats: A test coverage tool for RESTful APIs. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 594–598.

[15] Davide Corradini, Amedeo Zampieri, Michele Pasqua, Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2022. Automated black-box testing of nominal and error scenarios in RESTful APIs. *Software Testing, Verification and Reliability* (2022), e1808.

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL]

[17] Antonio Gamez-Diaz, Pablo Fernandez, Antonio Ruiz-Cortés, Pedro J Molina, Nikhil Kolekar, Prithpal Bhogill, Madhurranjan Mohaan, and Francisco Méndez. 2019. The role of limitations and SLAs in the API industry. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1006–1014.

[18] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. 2020. Intelligent REST API data fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 725–736.

[19] Google. [n. d.]. Google Maps Platform Documentation Google for Developers — developers.google.com. https://developers.google.com/maps/documentation.

[20] Gestao Hospital. 2022. Gestao Swagger Specification. https://github.com/ValchanOficial/GestaoHospital. [Online; accessed 19-August-2022].

[21] Myeongsoo Kim, Davide Corradini, Saurabh Sinha, Alessandro Orso, Michele Pasqua, Rachel Tzoref-Brill, and Mariano Ceccato. 2023. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1232–1243. doi:10.1145/3597926.3598131

[22] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2023. Adaptive REST API Testing with Reinforcement Learning. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 446–458.

[23] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: Model-based RESTful API Testing with Execution Feedback. *arXiv preprint arXiv:2204.12148* (2022).

[24] Alberto Martin-Lopez, Sergio Segura, Carlos Müller, and Antonio Ruiz-Cortés. 2021. Specification and automated analysis of inter-parameter dependencies in web APIs. *IEEE Transactions on Services Computing* 15, 4 (2021), 2342–2355.

[25] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2019. Test coverage criteria for RESTful web APIs. In *Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation*. 15–21.

[26] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2021. RESTest: automated black-box testing of RESTful web APIs. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 682–685.

[27] Phil McMinn. 2004. Search-based software test data generation: a survey. *Software testing, Verification and reliability* 14, 2 (2004), 105–156.

[28] Changhai Nie and Hareton Leung. 2011. A survey of combinatorial testing. *ACM Computing Surveys (CSUR)* 43, 2 (2011), 1–29.

[29] OpenAPI. 2023. OpenAPI Specification. https://www.openapis.org/

[30] Petstore. 2022. PetStore Swagger Specification. https://petstore.swagger.io/. [Online; accessed 19-August-2022].

[31] Stefano Russo. [n. d.]. Assessing Black-box Test Case Generation Techniques for Microservices. In *Quality of Information and Communications Technology: 15th International Conference, QUATIC 2022, Talavera de la Rina, Spain, September 12-14, 2022, Proceedings*. Springer Nature, 46.

[32] Stripe. [n. d.]. Error codes — stripe.com. https://stripe.com/docs/error-codes.

[33] Language Tool. 2022. Language Tool. https://github.com/languagetool-org/languagetool. [Online; accessed 19-August-2022].

[34] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]

[35] Chung-Hsuan Tsai, Shi-Chun Tsai, and Shih-Kun Huang. 2021. REST API Fuzzing by Coverage Level Guided Blackbox Testing. *arXiv preprint arXiv:2112.15485* (2021).

[36] Prasang Upadhyaya, Magdalena Balazinska, and Dan Suciu. 2016. Price-optimal querying with data apis. *Proceedings of the VLDB Endowment* 9, 14 (2016), 1695–1706.

[37] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTESTGEN: Automated black-box testing of RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 142–152.

[38] Yelp. [n. d.]. Getting Started — docs.developer.yelp.com. https://docs.developer.yelp.com/docs.

[39] Youtube. [n. d.]. YouTube Data API - Errors Google for Developers — developers.google.com. https://developers.google.com/youtube/v3/docs/errors.